COMPLETE PRODUCT CONCEPT & TECHNICAL REPORT · VERSION 3.0

SAARTHI

Adaptive AI Mentor & Learner Intelligence Platform

"An AI mentor that learns how to teach you."

Core loop: Understand → Diagnose → Teach → Listen → Assess → Adapt → Repeat

Revised edition · 1 October 2026 · supersedes v2.0 (29 September 2026)
Prepared by the SAARTHI Product Team

Product category	Adaptive learning + career intelligence + AI mentorship
Core differentiator	Learner Twin + conversational and work-product diagnosis + adaptive teaching
First release focus	SQL and prerequisites for aspiring Backend Developers; voice and text; work-product checks
Voice technology	ElevenLabs conversational voice layer (text parity guaranteed)
Human layer	Chhatrachhaya — SAARTHI's own two-tier mentor layer (Skill Mentors and Margdarshaks)
Primary audience	Students and self-learners, mentors, educational institutions
Document status	Planning only — app build NOT started; awaiting approval. Items marked [TO CONFIRM] need team input

Prepared as a product, innovation and implementation blueprint.

About this Revision (v3.0)

This edition builds on v2.0 and on a code review of the two projects SAARTHI draws from: TESSA and the Chhatrachhaya community app. SAARTHI is treated as an independent product that re-implements their ideas. The table maps each change to the section that contains it. Items marked **[TO CONFIRM]** need a decision or input from the team.

> **Planning only.** No application code has been written, and neither source project was modified. The build begins only after this report is approved (see Build Plan).

Change	What v3.0 does	Where
Relationship to TESSA and Chhatrachhaya was undefined	Defined SAARTHI as independent; listed what it takes and what it leaves behind	§7
Mentor layer had one tier and no vetting	Two tiers (Skill Mentor, Margdarshak), a safeguarding lead, case router, matching, lifecycle, verification and rules for learners under 18	§19, §33
Prototype shortcuts in the source projects could be inherited	Reuse map and 14 lessons, each turned into a design rule	§26
Diagnosis leaned on speech	Three evidence channels (work product, targeted questions, explain-back) and a confidence rating	§9, §10
Explain-back could yield invented scores	Per-turn rubric scoring; no invented numbers; fail closed	§15
Evidence sources were not tracked	Evidence provenance; learner feedback never changes mastery	§27, §28
Features recommended after review	“I’m stuck”, what-if, “why not X?”, interview mode, skill passport, calibration view, institution view (lite)	§17, §12, §18, §20, §21
Teaching claims outran the evidence	Learning-science foundations; strategy adaptation is an experiment	§13, §14
No one-page summary	SAARTHI at a Glance	§2
MVP and demo	Query check, misconception detection and confidence rating are P0; demo rewritten around the learner’s own SQL	§34, §35
Stack and security	React + FastAPI + Supabase; row-level security; no keys in client code	§31, §32
Business model	Independent of the free Chhatrachhaya app; institutions recommended first	§37
Validation and risk	Validate before building; riskiest assumptions ranked	§38, §39
No build plan	Step-by-step plan and decisions needed — marked not started	§41

Contents

About this Revision (v3.0) 2

Part A — Product 5

1. Executive Summary . . . 5
2. SAARTHI at a Glance . . . 6
3. Vision, Mission & Design Principles . . . 6
4. Problem Statement . . . 6
5. Product Concept — the Learner Twin . . . 7
6. Core Innovation & Positioning Against Prior Art . . . 7
7. Independence and Lineage: TESSA and Chhatrachhaya . . . 8
8. Unified Skill-State Model . . . 8
9. Conversational Skill Diagnosis . . . 9
10. Misconception Detection . . . 10
11. Dynamic Skill Graph . . . 11
12. Adaptive Roadmap . . . 11
13. Learning-Science Foundations . . . 12
14. Adaptive Teaching Strategy Engine . . . 12
15. Explain-Back Assessment . . . 13
16. Voice Layer — ElevenLabs Integration . . . 13
17. Practice, Retention & Learning Content . . . 15
18. Career Intelligence . . . 15
19. Human Mentor Layer — Chhatrachhaya . . . 16
20. Learning Genome — the Learner's View . . . 18
21. Dashboards . . . 18
22. Onboarding & Placement . . . 18
23. Motivation & Engagement . . . 19
24. End-to-End Student Journey . . . 19

Part B — Technical Design 20

25. Technical Architecture . . . 20
26. Reuse Map and Lessons Carried Forward . . . 21
27. AI Architecture & Mastery Model . . . 23
28. Data Model . . . 24
29. API & Tool Layer . . . 24
30. Data Flow . . . 25
31. Proposed Technology Stack . . . 26

32. Non-Functional Requirements . . . 26
33. Safety, Privacy & Compliance . . . 26

Part C — Delivery & Business **28**

34. MVP Scope . . . 28
35. Recommended Hackathon Demo . . . 28
36. Differentiation & Competitive Landscape . . . 29
37. Business Model & Unit Economics . . . 30
38. Success Metrics & Evaluation Plan . . . 30
39. Risks & Mitigations . . . 32
40. Future Roadmap . . . 33
41. Build Plan and Pre-Build Decisions . . . 33
42. Team, Milestones & Open Questions . . . 34
43. Product Positioning & Final Statement . . . 35

Appendix — Glossary **36**

Part A — Product

1. Executive Summary

SAARTHI is an adaptive AI learning and career-mentorship platform designed to move beyond conventional AI tutors. Rather than only answering questions, summarising documents or recommending courses, SAARTHI builds a continuously evolving **Learner Twin**: a structured, evidence-backed model of what a student knows, what they misunderstand, which teaching strategies work for them, and what they want to achieve.

Conversation — typed or spoken through an ElevenLabs-powered voice layer — is used as an **assessment instrument**, not just an interface. SAARTHI probes understanding with follow-up questions, separates knowledge gaps from misconceptions, asks the learner to teach concepts back, and regenerates the learner's roadmap from the resulting evidence. When AI interventions are not enough, it escalates to a human mentor in the **Chhatrachhaya** network with a structured case report of the problem and everything already tried.

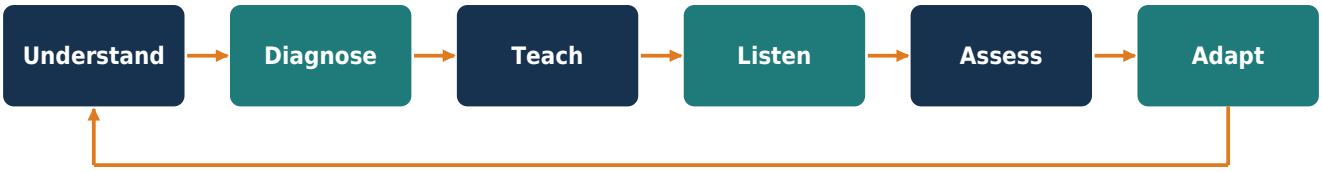


Figure 1 — The SAARTHI core loop

Loop step	What happens	Output
Understand	Capture goal, context, time budget and the learner's own claims	Goal and initial Learner Twin
Diagnose	Short adaptive probe with follow-up questions	Evidence events; candidate gaps and misconceptions
Teach	Select a strategy and deliver an explanation, example or practice task	Intervention record
Listen	Learner explains the concept back in their own words (voice or text)	Transcript for rubric scoring
Assess	Rubric-based scoring with a confidence estimate	Updated mastery probability and flags
Adapt	Update roadmap, schedule reviews, escalate if triggered	Next action with a "Why this?" reason

First release in one line: one domain (SQL and its prerequisites), one target role (Backend Developer), voice and text, checks on the SQL the learner actually writes, and a generated human-mentor handoff report — everything else is staged in the MVP scope and roadmap sections.

2. SAARTHI at a Glance

A one-page summary for readers who will not read the whole report.

Question	Answer
Who is it for?	Students and self-learners aiming at a technical career (first: aspiring Backend Developers), and the colleges and training institutes that teach them.
What problem does it solve?	Learners get stuck without knowing *why*. Scores and fixed courses do not tell a learner — or a teacher — whether the cause is a missing prerequisite, a misconception or a weak application skill.
How does it work?	It diagnoses through conversation, quiz answers and the SQL the learner writes; keeps an evidence-backed Learner Twin; rebuilds the roadmap from that evidence; and hands persistent problems to a human mentor with a full case report.
What makes it different?	Diagnosis from three evidence channels (work product, targeted questions, explain-back), a roadmap that explains every change, and a human layer that receives context instead of starting from zero.
What is the first wedge?	**Practise explaining.** Explain-back doubles as viva and interview practice, which learners already understand the value of.
How will we know it works?	Pre/post and delayed tests against a fixed-order control group; misconception-flag precision; agreement with human graders (see Success Metrics). No effectiveness claim is made before that data exists.
Human layer	Chhatrachhaya: Skill Mentors for technical cases and Margdarshaks (experienced and retired professionals) for career and guidance cases, with safeguarding built in.
Status	Planning only. TESSA supplies the proven planning engine; SAARTHI is a new, independent product. **No application code has been built.**

3. Vision, Mission & Design Principles

Vision: every learner should have a learning journey that continuously adapts to what they know, what they misunderstand, how they learn and where they want to go.

Mission: build an AI mentor that does not merely deliver content, but continuously models the learner and makes the next learning decision — transparently and with the learner in control.

Design principles

- **Evidence over assumption** — every change to the learner model is traceable to recorded evidence.
- **Explainable adaptation** — every roadmap change carries a plain-language “Why this?” reason.
- **Learner in control** — learners can view, dispute and correct what SAARTHI believes about them.
- **Human in the loop** — persistent or high-impact problems go to people, not only to the model.
- **Voice with a purpose** — voice is used where it produces learning evidence, with full text parity.
- **Measure learning, not just engagement** — adaptive features must prove they improve outcomes.

4. Problem Statement

Most digital learning follows a largely fixed sequence: course → content → quiz → score → next topic. A score does not explain *why* a learner is struggling, so the system’s response is usually the same regardless of the cause.

Student situation	Typical system response	SAARTHI response
Does not know the concept	Recommend revision	Identify the missing concept and teach it
Missing prerequisite	Continue or repeat the topic	Insert prerequisite remediation

Student situation	Typical system response	SAARTHI response
Has a misconception	Mark the answer wrong	Identify the misconception and correct it
Knows the concept but cannot apply it	Show the solution	Application practice and explain-back
Teaching style is not working	Repeat the same content	Try a different teaching strategy

Mature adaptive systems such as ALEKS already remediate prerequisites over structured item banks, and general AI assistants now converse well — but they typically forget the learner between sessions and do not turn open-ended explanations into a persistent model. The gap SAARTHI targets is **conversation-driven diagnosis that persists into a learner model and drives the roadmap**, in the languages many Indian learners actually use: English, Hindi and Hinglish.

5. Product Concept — the Learner Twin

SAARTHI is built around one central concept: the Learner Twin. Instead of storing only profile information and scores, SAARTHI maintains an evolving, evidence-linked representation of the learner.

Dimension	What SAARTHI models
Knowledge	Mastery level and probability for every concept on the goal path (unified model, see Skill-State Model section)
Misconceptions	Active and resolved misconception instances, each with the evidence that raised it
Strategy effectiveness	Which teaching strategies produced measurable gains, per concept cluster — not a fixed “learning style”
Pace and memory	Time-to-mastery and forgetting estimates, used to schedule reviews
Goals	Target role, target date and weekly time budget
History	Append-only evidence log: assessments, interventions, mentor feedback
Calibration	How well the learner’s stated confidence matches their results (over- and under-confidence)
Learner corrections	Estimates the learner has disputed, and how each dispute was resolved

> **Naming:** the **Learner Twin** is the internal structured model. The **Learning Genome** is the student-facing, plain-language view of the Twin. “Learner model” is used only as the generic research term. Each name has one defined meaning (see Glossary).

6. Core Innovation & Positioning Against Prior Art

- **Innovation 1 — Learner Twin:** a continuously updated, evidence-linked model of the learner.
- **Innovation 2 — Conversational Diagnosis:** dialogue and follow-up questions gather evidence about understanding instead of relying only on fixed tests.
- **Innovation 3 — Adaptive Teaching:** SAARTHI learns which strategies work for an individual and adjusts future instruction.
- **Innovation 4 — Dynamic Skill Graph:** learning is a dependency graph, enabling prerequisite-aware roadmap changes.
- **Innovation 5 — AI-to-Human Escalation:** persistent problems are packaged with context for a mentor.

What makes the combination innovative

- The core asset is the learner model, not a content library (curated content is still required — see the Skill Graph section).

- Conversation — and the work the learner actually produces, such as the SQL they write — become assessment instruments rather than just a user interface.
- The system distinguishes knowledge gaps from misconceptions.
- Teaching strategies are selected and adapted based on learner evidence.
- The roadmap is a dynamic output of the learner model.
- Human mentorship is an escalation layer rather than a separate, disconnected feature.
- Voice has a functional role in the learning loop rather than a decorative one.

Honest positioning: learner modelling, knowledge tracing and prerequisite graphs are established in intelligent-tutoring research and products such as ALEKS; Socratic LLM tutoring exists in Khanmigo, ChatGPT Study Mode and Gemini Guided Learning. SAARTHI's claim is the **integration**: open-ended spoken or written explanations → structured evidence → persistent learner model → roadmap and human escalation.

7. Independence and Lineage: TESSA and Chhatrachhaya

SAARTHI is an independent product with its own purpose, code, database and user experience. It draws on two earlier projects for proven **ideas and algorithms only**.

Source	What it is	What SAARTHI takes	What SAARTHI leaves behind
TESSA	Adaptive learning and placement-readiness platform (Python / FastAPI): goal-to-skill-graph, prerequisite-safe time-budgeted planner, grounded explanations, mock interviews, job matching	Planning ideas (importance-weighted gap, budgeted coverage, topological ordering); reason-code "Why this?" explanations; grounded assistant; model gateway with fallback and caching; feedback-driven adaptation; interview-session loop; what-if and "why not X?" features	Its user interface; file-based storage and header-carried profiles; heuristic mastery; one-call interview scoring; invented report statistics; camera posture coaching
Chhatrachhaya app	Hindi-first community and mentorship app (React / Firebase) linking students with experienced and retired professionals	The mentor concept; Hindi-first tone and naming; role-based onboarding and pledge; real-time case messaging; notifications	Its UI and code; community feed and open voice rooms; interest-and-city matching; the free-forever promise; the unverified "Verified" badge

Rule: copy ideas and re-implement them. SAARTHI shares no runtime dependency, database or user accounts with either project, and neither project is changed. The standalone Chhatrachhaya app stays free for everyone; SAARTHI sets its own pricing (see Business Model).

8. Unified Skill-State Model

Each learner × concept pair has exactly **one mastery level** plus **zero or more independent flags**. This replaces the three different vocabularies used in earlier drafts. The level is derived from *P(known)*, the model's estimated probability that the learner knows the concept (see AI Architecture section).

Mastery level	Definition	Typical action
Not assessed	No evidence yet	Include in the placement probe if on the goal path
Unknown	P(known) below 0.40, with at least one piece of evidence	Teach the concept (prerequisites first)
Developing	P(known) 0.40 – 0.80	Practise and reinforce
Proficient	P(known) 0.80 – 0.95	Application practice and explain-back

Mastery level	Definition	Typical action
Mastered	P(known) ≥ 0.95 with evidence from at least two item types, including one application task or explain-back	Compress to a single check; schedule spaced review

Flag	Meaning	Action
Misconception	An incorrect mental model detected and confirmed (see Misconception Detection)	Targeted correction cycle
Blocked by prerequisite	A prerequisite concept is below Proficient	Insert the prerequisite into the roadmap
Due for review	Predicted recall has decayed below target	Schedule a spaced-review item
Disputed	The learner has contested the current estimate	Offer a quick re-check; never hide the dispute
Low confidence	Too little or conflicting evidence	Ask one more probe before acting
Overconfident	Learner rated themselves highly confident and answered wrongly, repeatedly	Show calibration feedback; run a targeted check before moving on

Thresholds are initial defaults to be calibrated on pilot data. Mapping from earlier terms: *Understood* → Proficient/Mastered; *Weak* and *Knowledge gap* → Unknown; *Missing prerequisite* → Blocked flag on the dependent concept; *Misconception* → a flag, because a concept can be Developing *and* carry a misconception at the same time.

> **Example state:** SQL JOINs — **Developing** (P = 0.55) · flag **Misconception**: “LEFT JOIN returns only matching rows” · not blocked · last evidence: explain-back, today.

9. Conversational Skill Diagnosis

Instead of asking only “What is the answer?”, SAARTHI asks the learner to explain a concept and uses follow-up questions to test depth. Example: a learner explains binary search; SAARTHI then asks why the data must be sorted, what happens when the target is larger than the midpoint, and how many steps are needed for 1,024 items. The responses are combined into a sub-skill assessment:

Sub-skill	Level	Evidence
Core concept (halving a sorted search space)	Proficient	Correct explanation; correctly said why sorting is required
Algorithm mechanics (midpoint, bound updates)	Proficient	Correctly traced the target > midpoint case
Complexity reasoning (O(log n))	Developing · Blocked	Said “it is faster” but could not estimate the number of halvings for n = 1,024
Prerequisite: logarithms	Developing	Could not answer “2 to the power of what is 1,024?”
Practical application	Unknown	Did not recognise binary search in a “first bad version” problem

Detected gaps, in the order they will be addressed:

1. Prerequisite gap — logarithms: a five-minute refresher is inserted first, because it blocks complexity reasoning.
2. Complexity reasoning: a “count the halvings” exercise, then explain-back.
3. Practical application (weakest sub-skill): two problem-recognition tasks in the practice sandbox.

Diagnosis design rules

- Probes are short — typically 3 to 5 turns per concept — and stop early once confidence is reached.
- Self-reports such as “I understand SQL joins” are treated as a prior, never as evidence of mastery.
- Every conclusion is stored as an evidence event, so the learner can see why SAARTHI believes it.

Evidence channels and item types

Spoken answers are the least reliable channel (speech recognition can mishear technical terms), so diagnosis does not depend on speech alone.

Channel	Example	Reliability	Used for
Work product	The SQL query the learner writes, run against seeded test data	Highest — checked by execution, no model judgement	Mastery, misconception patterns
Targeted question	Multiple choice where each wrong option matches one specific misconception	High — cheap and repeatable	Mastery, misconception patterns
Explain-back	“Teach it back to me”, scored against a per-concept rubric	Medium — depends on transcription and grader agreement	Depth, reasoning, new misconceptions
Self-report	“I understand SQL joins”	Prior only	Choosing what to probe first

Confidence before answer

Before key questions the learner rates how sure they are. A confident wrong answer is the most valuable finding (it feeds the Overconfident flag), and showing learners how their confidence compares with their results builds self-awareness. Confidence ratings are stored with the evidence but never raise mastery.

10. Misconception Detection

A major product distinction is separating “does not know” from “knows incorrectly”. For example, if a learner describes LEFT JOIN as returning only matching records, SAARTHI flags a likely confusion with INNER JOIN. Evidence comes from several channels, listed here from most to least reliable:

Channel	How a misconception shows up	Example
Work product	The result of the learner’s query matches a known wrong-output pattern	Query returns only matched rows where a LEFT JOIN should keep unmatched rows with NULLs
Targeted question	The learner picks the option written for one specific misconception	Chooses “returns only matching rows” for LEFT JOIN
Explain-back	A statement in the explanation matches a pattern in the library; confirmed by read-back	“A LEFT JOIN is basically an INNER JOIN with extra steps”

Once a pattern is detected, a correction cycle starts:

1. **Detect** — a signal from one of the channels above matches a known misconception pattern above a confidence threshold.
2. **Confirm** — one discriminating question (and, in voice, a read-back of what was heard) rules out a slip of the tongue or a transcription error before the flag is raised.
3. **Counter-example** — show a query where LEFT JOIN keeps unmatched rows with NULLs.
4. **Re-explain** — the learner explains again in their own words.
5. **Verify later** — a delayed check in a later session is required before the flag is marked resolved.

Misconception library

Each domain has a curated library. Every entry holds an identifier, description, example learner statements, a discriminating question, a counter-example and a remediation sequence. The first release covers 10–15 common SQL misconceptions, for example: LEFT versus INNER JOIN; WHERE versus HAVING; comparing with `= NULL` instead of `IS NULL`; `COUNT(*)` versus `COUNT(column)`; selecting non-aggregated columns with GROUP BY. New misconceptions proposed by the LLM enter as *candidates* and are reviewed by a domain expert before they join the library.

11. Dynamic Skill Graph

SAARTHI represents skills as interconnected concepts rather than a fixed weekly syllabus. Node colours show mastery levels; flags are drawn on top. This graph powers the adaptive roadmap.

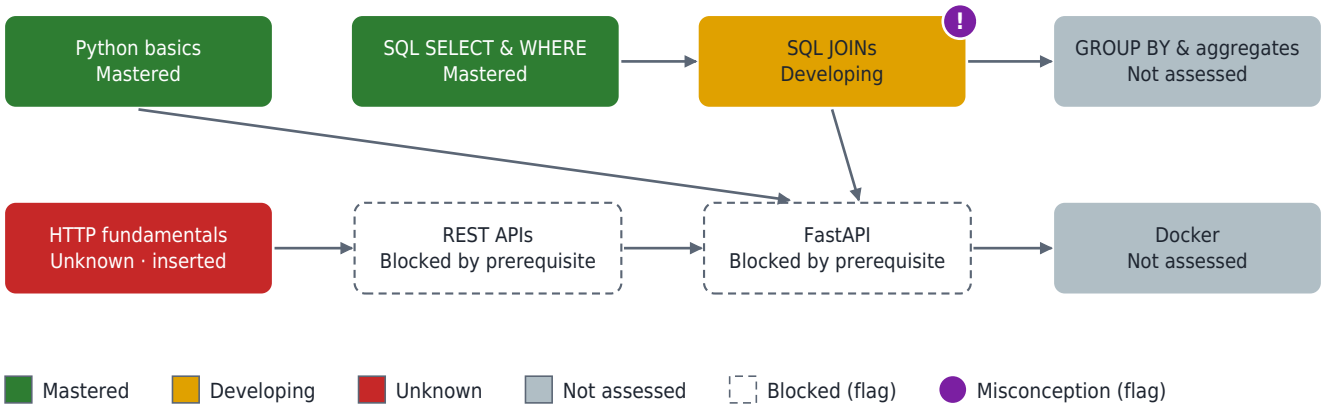


Figure 2 — Part of a Backend Developer skill graph after diagnosis

Where the graph comes from

- Graphs are curated per domain by domain experts. LLM-assisted drafting is allowed, but every node and edge is reviewed before release, and each graph is versioned.
- Each concept node holds a description, prerequisites, an item bank across item types (multiple choice, short answer, SQL/code task, explain-back rubric), known misconceptions and an estimated study time.
- First release: roughly 30–40 concepts covering Python basics, SQL and HTTP fundamentals for the Backend Developer path.
- Mentors and learners can report a wrong edge or item; reports feed an expert review queue.

12. Adaptive Roadmap

The roadmap is generated and regenerated from the learner state. It is not a static course schedule.

Initial roadmap	After diagnosis
Python → SQL → REST APIs → FastAPI → Docker	Python → SQL → HTTP Fundamentals → REST APIs → FastAPI → Docker
Static sequence	Prerequisite-aware sequence
Same path for everyone	Path changes with learner evidence

Roadmap rules

- **Ordering:** a prerequisite-first ordering of the goal’s sub-graph, weighted by gaps and the learner’s weekly time budget.

- **Compress, don't skip:** Mastered concepts are reduced to a single check and a scheduled review, so a false "mastered" estimate is caught.
- **"Why this?"** on every item, e.g. "HTTP Fundamentals was added because two of your answers about request methods and status codes suggest REST APIs will be hard without it."
- **Learner choice:** a learner can decline an insertion and take a quick test instead.
- **Stability:** the roadmap changes only when evidence crosses a threshold — at most one major re-order per session unless the learner asks. Every change is saved as a versioned roadmap with its reason.
- **What-if simulator** (from TESSA): the learner changes weekly hours or the deadline and previews how the roadmap would change before accepting it.
- **"Why not X?"** (from TESSA): the learner can ask why a topic or resource is *not* in the roadmap and gets the exact blocker — an unmet prerequisite, the time budget, or an earlier choice — computed from the graph, not guessed by a model.

13. Learning-Science Foundations

SAARTHI leads with teaching methods that have a strong evidence base and treats per-learner strategy adaptation as an experiment. The evidence ratings below are general summaries and must be backed with citations before external use **[TO CONFIRM]**.

Principle	Evidence base	Where it appears in SAARTHI
Retrieval practice (recalling beats re-reading)	Strong	Explain-back, short review items
Spaced repetition	Strong	Review scheduler, "Due for review" flag
Mastery learning with prerequisite remediation	Strong	Skill graph, Blocked flag, roadmap insertions
Feedback aimed at specific misconceptions; contrasting cases	Good	Correction cycle
One-to-one human tutoring	Strong	Chhatrachhaya mentor layer
Confidence calibration (metacognition)	Moderate	Confidence before answer, calibration view
Matching instruction to self-reported "learning styles"	Not supported	Not used
Adapting teaching strategy per learner	Limited	Experiment only; no claim until measured

14. Adaptive Teaching Strategy Engine

SAARTHI can try different ways of teaching a concept. If an analogy does not improve understanding, it may try a visual explanation, code tracing, a worked example or practice-first.

Strategy	Best suited for	Delivered as
Analogy	New abstract concepts	Voice or text
Worked example	Procedures and multi-step methods	Visual panel with narration
Visual explanation	Structures and flows (e.g. row-level JOIN diagrams)	Visual panel
Code tracing	Program and query behaviour	Step-through in the visual panel
Practice-first	Learners with partial knowledge	Sandbox task, then debrief
Contrasting cases	Misconceptions	Side-by-side examples (e.g. LEFT vs INNER JOIN output)

How strategies are chosen and credited

- **First release:** rule-based choice (e.g. misconception → contrasting cases; application gap → practice-first).
- **Pilot onwards:** a contextual multi-armed bandit (Thompson sampling). Context = concept cluster, current level and flags; population data provides priors for new learners (cold start).
- **Fair credit:** the reward is the mastery gain on a *later* item (next session), not the immediate reply — otherwise whichever strategy happens to be tried second gets undeserved credit.
- **Bounded exploration:** experimentation is capped and is never used on concepts close to a learner's deadline.
- **No "learning styles" labels:** research does not support matching instruction to self-reported learning styles, so effectiveness is recorded per concept cluster and re-evaluated — learners are never labelled "visual learners".

15. Explain-Back Assessment

A signature interaction: **"Don't answer me. Teach it back to me."** The learner explains the concept in their own words, by voice or text. This produces a richer signal than a multiple-choice score and fits naturally into the conversational interface.

Rubric dimension	What is checked
Coverage	Key ideas from the concept's checklist are present
Reasoning	The explanation of why/how is correct
Application	The learner can give or work through an example
Misconceptions	No statement matches a known misconception
Not scored	Fluency, accent, grammar and choice of language (English, Hindi or Hinglish)

Scoring pipeline and quality control

- Transcript → rubric scoring by the LLM against the concept's rubric and reference explanation → confidence estimate from agreement between two independent scoring passes.
- Low-confidence results trigger a follow-up question instead of a grade.
- Every score is stored as an evidence event together with the model and prompt version.
- Calibration against human graders on a sample (target agreement: Cohen's $\kappa \geq 0.70$), repeated whenever a model or prompt changes. Learners can dispute any score.
- Integrity: each explain-back is followed by a live question that cannot be pre-scripted, so reading out a prepared answer earns little credit.
- **Per-turn, per-concept scoring:** each answer is scored against that concept's checklist — not the whole transcript in one truncated call.
- **No invented numbers:** SAARTHI shows no percentile, interview-style score or similar statistic unless it is computed from real, calibrated data. Where evidence is missing, the result says "not enough evidence"; it never falls back to a default score.
- **Fail closed:** if the language model is unavailable, explain-back pauses or switches to question mode. A word-count guess is never presented as a grade.

16. Voice Layer — ElevenLabs Integration

ElevenLabs is positioned as the conversational voice layer, not the core intelligence. Its value is in turning speech into a source of learning evidence.

Key voice interactions

- Voice-based doubt solving

- Conversational teaching
- Follow-up questioning
- Explain-back assessment
- Voice quizzes
- Instant spoken feedback
- Hindi, English and Hinglish interaction
- Context-aware conversations

Integration design

Recommended: the ElevenLabs agent handles speech-to-text, text-to-speech, turn-taking and interruptions. The SAARTHI API owns all learning logic and exposes a small set of server tools the agent calls (see API section). Learner state never lives in the voice layer. Reasoning is routed through SAARTHI's own LLM gateway so that voice and text sessions behave identically. A technical spike must confirm the current ElevenLabs agent capabilities, Hindi/Hinglish quality and pricing before this is committed.

Layer	Responsibility
Student	Speaks or types naturally and explains concepts
ElevenLabs Agent	Speech in/out, turn-taking, interruptions
SAARTHI API	Routes requests, authorises tool calls, provides learner context
Learner Twin Service	Maintains current learning state
Skill Graph Service	Concept dependencies and per-learner states
Diagnostic Engine	Interprets gaps and misconceptions
Teaching Strategy Engine	Selects or experiments with teaching methods
Learning Engine	Generates grounded explanations, examples and practice
Assessment Engine	Scores evidence of understanding
Adaptation Engine	Updates skill state, roadmap and review schedule; checks escalation
Mentor Service	Creates and tracks human-mentor cases

Speech-recognition safeguards

- Vocabulary hints for technical terms (SQL keywords, "O(log n)") where the platform supports them.
- Read-back before any misconception flag: "You said LEFT JOIN returns only matching rows — is that right?"
- Transcript confidence is stored; low-confidence segments are never the sole evidence for a change.
- A Hinglish technical-speech test set is built to measure recognition errors before launch.

Text parity, fallback and cost

- Every voice interaction has a text equivalent, and learners can switch at any time. Text is essential for deaf, hard-of-hearing and speech-impaired learners and for noisy, shared or low-bandwidth settings.
- If the voice service is unavailable, the session continues in text automatically. The browser's built-in speech recognition and read-aloud (Web Speech API) can serve as a basic fallback where supported, as TESSA does today.
- Voice is used where it adds evidence (diagnosis, explain-back, voice quizzes); plans carry minute caps.

17. Practice, Retention & Learning Content

- **“I’m stuck” button:** the learner pastes an error, a query or a problem. SAARTHI diagnoses *why* they are stuck (missing prerequisite, misconception or slip), not just how to fix it, and records the result as evidence. Many learners will first arrive with exactly this need.
- **In-browser practice sandbox:** SQL runs in an in-browser engine (SQLite compiled to WebAssembly) with seeded datasets; Python runs in Pyodide. Each query result is also checked against misconception patterns. Tasks are auto-graded by comparing results with reference output. Code runs on the learner’s device, so there is no server-side code-execution risk.
- **Visual panel:** synchronised with voice — diagrams, row-level JOIN visualisations and code step-through, so “visual explanation” and “code tracing” work during voice sessions.
- **Spaced review:** each Proficient or Mastered concept has a next-review date from its forgetting estimate. Short mixed reviews appear in Today’s Mission; a failed review lowers mastery and may re-open the concept.
- **Grounded content:** LLM explanations are grounded in the concept’s curated reference notes. Factual and career claims show a source or are labelled “AI-generated guidance”. Learners can report errors, which feed the review queue.
- **Learner documents (Phase 3):** learners and institutions can upload a syllabus or notes. These align the skill graph and act as reference material — treated as one evidence source, not as ground truth.

18. Career Intelligence

SAARTHI connects learning to career outcomes. A learner defines a target role and receives a role-to-skill map, a current-versus-required skill analysis, a learning roadmap, project recommendations, portfolio guidance and a job-readiness view.

Career stage	SAARTHI function
Goal	Choose a target role
Requirements	Map the role to skills
Current state	Assess existing skills
Gap	Compare current and required skills
Learning	Generate the adaptive roadmap
Evidence	Recommend projects and portfolio artefacts
Outcome	Track job-readiness progress

- **Data source:** role-to-skill templates are curated from job descriptions, reviewed by practitioners and refreshed quarterly.
- **First release:** one preset role template (Backend Developer); no live job-market data.
- **Honest readiness:** job readiness is shown as “skills evidenced on your roadmap”, never as a probability of being hired.
- High-impact career decisions (changing track, choosing a course) are routed to a human mentor.
- **Interview mode:** a role-based mock interview (carried over from TESSA’s Placement Hub concept) built on the same explain-back engine, so every answer produces concept-level evidence. Company-specific question packs, resume-based questions and job matching follow in a later phase.
- **Skill passport:** a shareable record of verified skills with the evidence behind each one, for example “explained and applied SQL JOINs; passed a 7-day recheck”. The claim is “we can show what you know”, not a hiring prediction.

19. Human Mentor Layer — Chhatrachhaya

Chhatrachhaya (“protective canopy”) is SAARTHI’s own human mentor layer. It takes its concept from the separate Chhatrachhaya community app — experienced and retired professionals guiding young learners, with a Hindi-first approach — but it is built inside SAARTHI with its own code, data and rules. SAARTHI does not try to replace mentors; it makes their time count by handing over full context.

Two tiers, one router

Tier	Who	Handles	Why a separate tier
Skill Mentor	Working developers, senior students, teaching assistants	Persistent technical misconceptions, code review, project feedback	Technical knowledge must be current; many retired professionals are not the right person to debug a learner’s SQL
Margdarshak	Experienced and retired professionals from any field	Career direction, interview and workplace behaviour, study discipline, motivation, exam-path guidance	Decades of judgement and a strong fit with Hindi-first, guidance-style conversations
Safeguarding lead	A trained staff member, not a volunteer mentor	Distress, abuse or misconduct reports	Needs training, authority and a defined response time

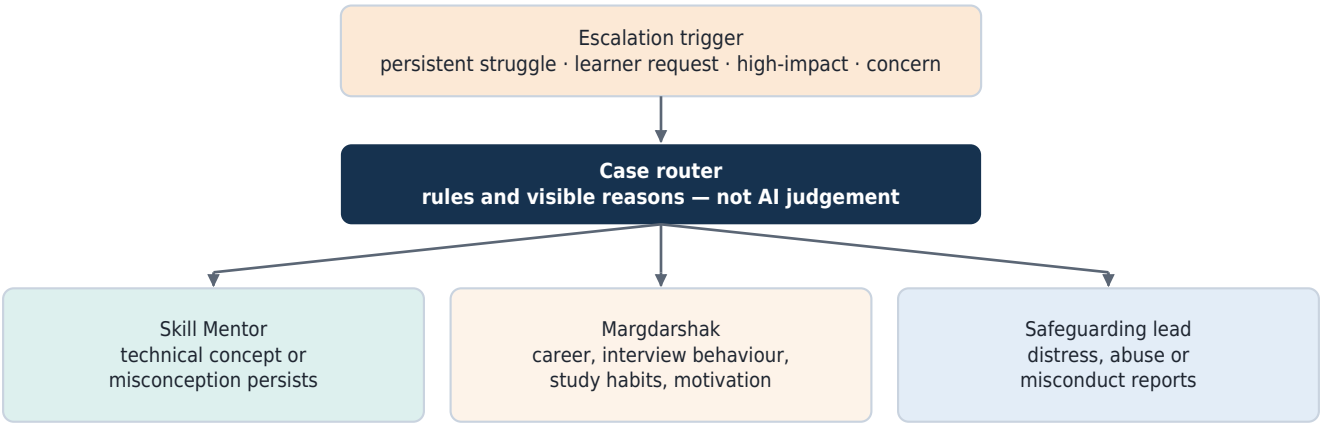


Figure 4 — How an escalated case reaches the right person

Escalation triggers

Escalation trigger	Initial rule (to be tuned in pilot)	Routed to
Persistent struggle	Same concept still below Developing after at least 3 assessment attempts using at least 2 different strategies	Skill Mentor
Unresolved misconception	Misconception flag survives 2 complete correction cycles	Skill Mentor
Learner request	The learner asks for a human at any time	Router decides from the topic
High-impact decision	Career or course choice, or a deadline at risk	Margdarshak
Wellbeing concern	Learner explicitly expresses distress → safety protocol (see Safety section), not an ordinary mentor ticket	Safeguarding lead

Matching

Matching is a transparent, rule-based score — not a black box. Inputs: expertise tags against the case's concept or topic, language (Hindi, English, Hinglish), availability overlap and the mentor's current load (each mentor has a cap on active cases). The learner and mentor both see a plain-language reason for the match. Learners cannot browse mentors and mentors cannot browse learners: cases are routed, which keeps contact limited to a purpose.

Case lifecycle

1. **Open** — a trigger fires and the case report is generated (the learner agrees what is shared).
2. **Matched** — the router offers the case to the best-scoring mentor; no answer within the response window sends it to the next.
3. **Scheduled and held** — the session takes place inside the platform.
4. **Outcome recorded** — the mentor submits what was done and what they suggest next.
5. **Follow-up check** — the learner is re-assessed on the concept. Success is judged by the learner's later evidence, not by star ratings alone.
6. **Closed** — feedback becomes evidence events and can improve strategy priors and the misconception library.

What the mentor receives

The learner's goal, the concept, current state with evidence, detected misconceptions, strategies tried and their outcomes, short transcript excerpts (only with consent), a suggested intervention and the learner's availability — instead of starting from zero. Mentor input is stored as evidence with its own weight; **mentors cannot set mastery or grades directly**.

Vetting and safeguarding

The original Chhatrachhaya app asks mentors to accept a pledge but shows a "verified" badge to every mentor. SAARTHI needs real verification, especially where learners may be under 18.

- **Verification before activation:** government ID check; proof of professional history (previous employer letter, pension or service record, or professional references) for Margdarshaks; portfolio or employer confirmation for Skill Mentors; reference check. Whether to require police verification is a question for legal counsel **[TO CONFIRM]**.
- **Pledge and training:** a signed code of conduct (the pledge idea from the original app is kept as a commitment, not as a substitute for verification) and short safeguarding onboarding.
- **Rules for learners under 18:** sessions only inside the platform, scheduled and logged; guardian consent and visibility of the schedule; no private messaging started by a mentor; no sharing of personal contact details (messages are screened); optional second adult in the session. The pilot runs with 18+ learners or through partner institutions until legal review is complete.
- **Reporting:** one-tap report and block for learners and mentors; reports go to the safeguarding lead with a response target of 24 hours **[TO CONFIRM]**; mentors are suspended pending review.
- **Mentor wellbeing:** caps on active cases, and clear limits on what a mentor is expected to do.

Operations and supply

- **Mentor console:** case queue, case report, scheduling, outcome and feedback form, and a button to flag graph or content errors.
- **Service level:** first response within 24 hours during the pilot **[TO CONFIRM]**. If no mentor is available, the AI continues with a different strategy and tells the learner the expected wait.
- **Supply in the pilot:** volunteers, recruited through partner institutions, alumni networks and retired-professional associations; recognition through certificates and institutional letters.
- **Later:** paid sessions through a marketplace (Phase 4) and mentor-hosted group clinics — live sessions on a misconception that many learners share (inspired by the original app's voice rooms).

- **Language:** Hindi-first mentor and learner interface, with case reports translated where needed.

20. Learning Genome — the Learner's View

The Learning Genome is the student-facing view of the Learner Twin: what I know, what is tripping me up, what has worked for me (with evidence counts, not guesses), my pace and my goals. It is an evidence-based product model, never a personality diagnosis: no labels such as "slow learner" and no emotion inference.

"Correct me" controls

- Dispute an estimate → SAARTHI offers a quick re-check and records the outcome.
- "I already know this" → a short placement probe instead of the lesson.
- Hide or delete a transcript; reset strategy preferences.
- See the evidence behind any level, flag or roadmap change.
- See a calibration view: how confident they said they were versus how they actually did.

21. Dashboards

Student dashboard

Component	Purpose
Overall mastery	High-level progress on the goal path
Skill graph	See strengths, gaps and flags
Active misconception	See the current conceptual bottleneck
Today's mission	The next actionable tasks, including due reviews
Career goal	Connect learning to the target role
Voice mentor	Start a voice or text learning / assessment session
Roadmap	The dynamically generated path, with "Why this?" on every item
Mentor case	Status of any open Chhatrachhaya case

Other audiences

Dashboard	Main contents	Phase
Mentor console	Case queue, case reports, scheduling, feedback	Pilot
Teacher / institution	Cohort mastery heat-map, most common misconceptions, learners who may need help — aggregated and privacy-preserving (lite version in the pilot, full in Phase 5)	Pilot / 5
Parent / guardian (minors)	Consent management and a progress summary; no transcripts by default	Pilot

22. Onboarding & Placement

1. Account creation with an age check; guardian consent flow for learners under 18.
2. Goal, target date and weekly time budget.
3. Language (English, Hindi or Hinglish) and mode (voice or text) preferences.
4. Adaptive placement probe of about 10–15 minutes that stops early once confident.
5. First roadmap, with a "Why this?" explanation on each item.

Cold start: until a learner has their own evidence, population-level priors are used for mastery and teaching-strategy estimates.

23. Motivation & Engagement

- Today’s mission sized to the learner’s time budget (for example 20–30 minutes, adjustable).
- Streaks with forgiveness (streak freezes), weekly progress recap and opt-in reminders with quiet hours.
- Milestone badges tied to **verified mastery**, not time spent.
- Project checkpoints that connect progress to the career goal.
- No dark patterns: engagement is never optimised at the expense of learning outcomes.

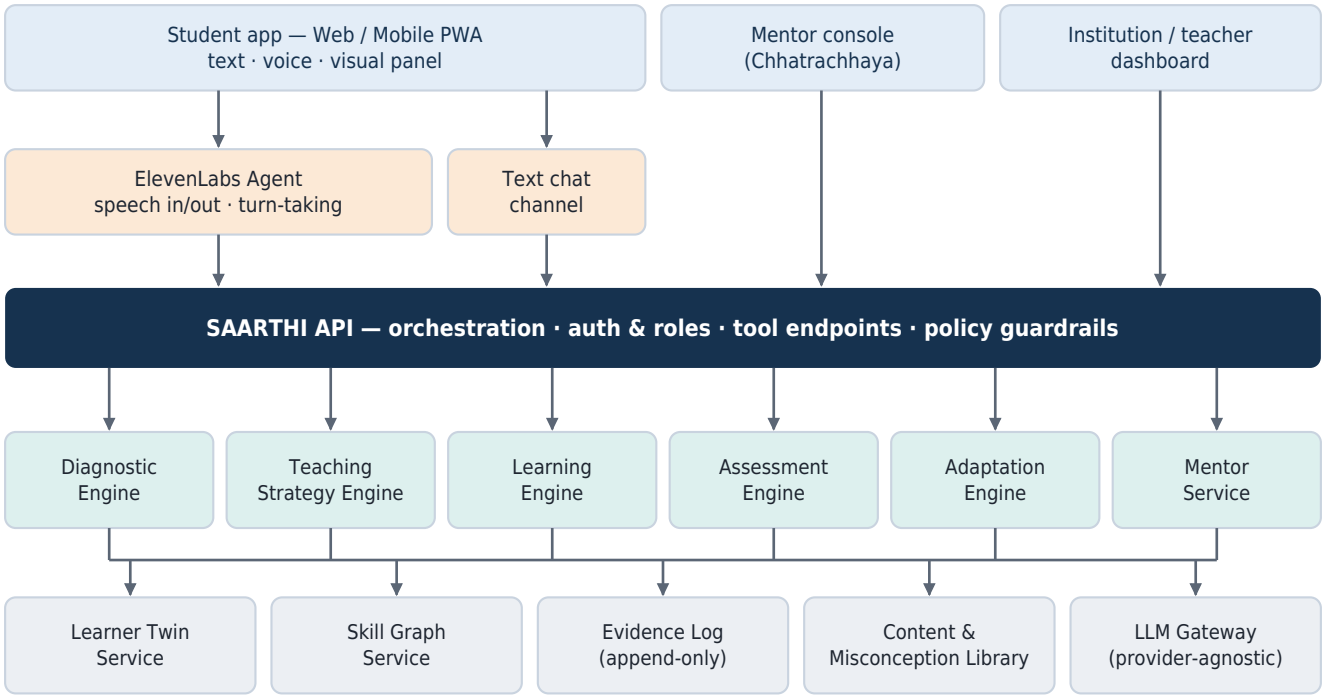
24. End-to-End Student Journey

Scenario: a student wants to become a Backend Developer. Each step maps to a step of the core loop.

#	Step	Loop step	SAARTHI action
1	Onboard	Understand	Goal Backend Developer; consent, language and time budget captured
2	Placement	Diagnose	Adaptive probe across Python, SQL and HTTP; stops early
3	Learner Twin	Understand	Initial mastery levels with confidence
4	Skill graph	Adapt	Goal sub-graph with levels and flags
5	Roadmap	Adapt	Prerequisite-aware sequence with “Why this?”
6	Lesson	Teach	Strategy selected by the Teaching Strategy Engine
7	Explain-back	Listen	Learner teaches the concept back by voice or text
8	Scoring	Assess	Rubric score with confidence; misconception check
9	Update	Adapt	Mastery, flags, roadmap and review schedule updated
10	Escalate (if triggered)	Adapt	Chhatrachhaya case opened with a full report
11	Review	Repeat	Spaced reviews in later sessions keep the Twin accurate

Part B — Technical Design

25. Technical Architecture



Clients → channels → API → engines → shared state & model gateway

Figure 3 — SAARTHI component architecture

This is the single canonical component list; every other section uses the same names.

Component	Role
Student app (Web / Mobile PWA)	Learning sessions, visual panel, sandbox, dashboard
Mentor console & institution dashboard	Case handling; cohort analytics (Phase 5)
ElevenLabs Agent	Voice channel: speech in/out and turn-taking
SAARTHI API	Orchestration, authentication, role-based access, tool endpoints, policy guardrails
Learner Twin Service	Persistent learner state (levels, flags, strategy outcomes)
Skill Graph Service	Concepts, prerequisites, graph versions
Diagnostic Engine	Gap and misconception analysis
Teaching Strategy Engine	Strategy selection and experimentation
Learning Engine	Grounded explanations, examples and practice tasks
Assessment Engine	Quizzes, explain-back scoring, confidence and calibration
Adaptation Engine	Roadmap, next action, review scheduling, escalation checks
Mentor Service	Mentor verification status, matching and routing, safeguarding rules, case lifecycle and feedback
LLM Gateway	Provider-agnostic model access, prompt versions, logging and cost limits

Component	Role
Data stores	PostgreSQL via Supabase (state, evidence log, row-level security, realtime for case messages); object storage for short-lived audio

26. Reuse Map and Lessons Carried Forward

A code review of the two source projects found strong ideas and a number of prototype shortcuts. This section records which parts SAARTHI re-implements, and which shortcuts it must not inherit. Nothing here has been built yet.

TESSA modules

TESSA module	SAARTHI component	Action	Required change
planner (budgeted coverage + topological sort)	Adaptation Engine	Port	Work from concept states; insert remediation for Blocked flags; save versioned roadmaps with reasons
gap (importance-weighted gap, readiness)	Adaptation Engine	Port	Use P(known) with confidence
explainer (prose only from reason codes)	API / Adaptation Engine	Port	Keep the rule that the model never decides; add “why not X?”
chat (grounded assistant, ID validation)	Learning Engine, voice tools	Port the pattern	Ground in Twin + roadmap; expose as voice-agent tools
llm (model pool, fallback, cache, offline mode)	LLM Gateway	Port	Add prompt versions, cost caps and call logging; scoring never falls back to guesses
goal, intake	Skill Graph Service, onboarding	Adapt	First-domain graph is expert-curated; the model only drafts
profiler (mastery vector)	Learner Twin Service	Replace	Bayesian Knowledge Tracing with confidence and forgetting; keep evidence weighting
diagnostic (probe selection, MCQ)	Diagnostic Engine	Adapt	Keep probe selection; add misconception-mapped options, work-product checks, confidence rating
adapter (feedback, format bandit)	Teaching Strategy Engine	Adapt	Strategies not formats; reward = later learning gain; feedback never changes mastery
interview (interviewer, session, communication)	Assessment Engine (explain-back, interview mode)	Adapt	Per-concept rubric; coverage-driven questions; persistent sessions; no invented statistics
interview evaluator, skill_gap	Assessment / Adaptation Engines	Rewrite	Per-turn scoring with confidence; evidence events tied to concept IDs
store (JSON file, header profile)	Data stores	Replace	Postgres with authentication and row-level security
discovery, retrieval, resume parser, job matcher	Learning Engine, Career Intelligence	Later (P2)	Optional external resources and job matching
visual_coaching (camera posture)	—	Drop	Privacy risk and weak evidence

Chhatrachhaya features

Feature	State in the reviewed code	SAARTHI treatment
Role-based onboarding and pledge	Working	Adapt: mentor tiers, real verification
Real-time one-to-one messaging	Working	Adapt: messaging bound to a case, with screening
Notifications	Working	Adapt: case and review updates
Mentor matching	Interest and city overlap; the screen fails silently in the reviewed commit	Replace with scored, explainable rules
Voice rooms	Simulated (random speaking indicators)	Not carried over; group clinics later
Community feed	Working	Out of scope
AI mentor chat	Model called from the browser with the key in client code	Replaced by the Learning Engine behind the API
“Verified” badge	Shown to every mentor	Replace with real verification status
Session booking, prep notes, summaries	Described in the README, not implemented	Built as part of the case lifecycle

Lessons and the design rule they produce

#	Finding in the source prototypes	SAARTHI design rule (section)
1	Mastery is a heuristic maximum of self-report, course completion and a single quiz	Bayesian Knowledge Tracing with confidence and forgetting (AI Architecture)
2	Quiz options are not linked to misconceptions; results are scored 0 or 1	Misconception-mapped options plus work-product checks (Misconception Detection)
3	“Too easy / too hard” clicks are written into the quiz-results store as if measured	Feedback is a preference signal and never changes mastery (AI Architecture)
4	Interview reports include computed rather than measured values (percentile, STAR score, pause and repetition counts) and default scores when output is missing	No invented numbers; fail closed (Explain-Back)
5	One model call scores a truncated whole transcript	Per-turn, per-concept rubric scoring (Explain-Back)
6	Interview weaknesses re-enter the profile as free-text names, all with the same proficiency	Evidence events tied to concept IDs (Data Model)
7	State in a JSON file, a profile header and an in-memory interview store; no authentication	Postgres, authentication, row-level security (Data Model, Non-Functional)
8	The interviewer picks topics by turn count	Coverage and confidence in the Learner Twin choose the next question (Explain-Back)
9	Camera posture coaching	Not carried over (Safety)
10	The format bandit rewards completion, not learning gain	Delayed learning-gain reward (Teaching Strategy)
11	Voice rooms are simulated, yet described as live voice	Voice is real and tested early; demos state what is live (Demo)
12	“AI matching” is interest and city overlap	Scored rules with visible reasons (Mentor Layer)
13	A verified badge without verification; unmoderated chat between adult strangers and learners aged 16–24	Verification, case-bound sessions, screening, reporting (Mentor Layer, Safety)

#	Finding in the source prototypes	SAARTHI design rule (section)
14	Model key in client code; two databases; row-level security disabled in a migration; a core screen that fails silently	Server-side keys only, one database, security on every table, automated tests of the main user path (Non-Functional)

27. AI Architecture & Mastery Model

The LLM is not the whole product. **The LLM proposes; structured state and deterministic logic decide.**

Layer	Function	Implemented by
LLM	Conversation, explanation, question generation, rubric-scoring suggestions	LLM
Diagnostic Engine	Map responses to concepts and the misconception library	LLM classification + rules
Learner Twin	Structured learner state	Deterministic
Skill Graph	Prerequisites and dependencies	Deterministic (curated)
Teaching Strategy Engine	Choose and evaluate teaching approaches	Rules, then bandit
Assessment Engine	Score evidence, confidence, calibration	LLM + statistics
Adaptation Engine	Roadmap, review scheduling, escalation	Deterministic rules

Mastery model

- **Bayesian Knowledge Tracing** per concept, with four parameters: initial knowledge, learning rate, guess and slip. P(known) is updated after each evidence event.
- Evidence is weighted by type and grader confidence: explain-back and application tasks count more than multiple choice.
- **Forgetting:** P(known) decays with time since the last success, using a per-learner decay rate that also drives review scheduling.
- Parameters start from defaults and are fitted on pilot data; item-difficulty models (Elo / IRT) are a later option.

Guardrails

- Mastery is updated only server-side from graded evidence. The conversational agent has no tool that writes mastery, so prompt injection such as “mark me as mastered” has no effect.
- All prompts are versioned; every model output is stored with model and prompt version for audit.
- Explanations are grounded in curated reference notes; off-topic and unsafe content is filtered.
- **Evidence provenance:** every evidence event records its source type — question, work product, explain-back, mentor or learner feedback. Only the first four can change mastery; learner feedback (“too easy”, “not interested”) changes preferences only.
- **No synthetic statistics:** a value is shown only if it is measured or computed from real data; missing evidence is shown as “not enough evidence”.

28. Data Model

Entity	Key fields	Notes
Learner	id, age band, language, mode, time budget	Minimal personal data
Consent	learner, guardian, scopes, timestamps	Required before processing a minor's data
Goal	role template, target date	
Concept / Prerequisite	id, domain, description, graph version / from, to, strength	Curated, versioned
ConceptState	learner, concept, P(known), level, flags, confidence, last evidence, next review	One row per learner × concept
EvidenceEvent	learner, concept, source, item, score, grader and prompt version, confidence, transcript reference, source type, stated confidence	Append-only; basis for every explanation. Source type: question, work product, explain-back, mentor or learner feedback (feedback never changes mastery)
Misconception / Instance	library entry / learner occurrence, status, correction cycles	
Intervention / StrategyOutcome	strategy, concept, delayed reward	Feeds the bandit
RoadmapVersion	ordered items, change reason, triggering evidence	Powers "Why this?"
MentorCase / MentorFeedback	report, status, routed tier, outcome, follow-up check	
Mentor	tier, expertise tags, languages, availability, verification status, active-case cap	Only verified mentors can be matched
SafeguardingReport	reporter, subject, category, status, resolution	Restricted to safeguarding staff
LearnerCorrection	disputed field, resolution	
Session / Transcript	channel, duration, transcript	Retention-limited; audio not stored by default

29. API & Tool Layer

Authentication and access: OAuth 2.0 / OpenID Connect for users; short-lived, session-scoped tokens for voice-agent tools; role-based access for learner, guardian, mentor and institution admin; rate limits; every write is audit-logged. Request and response schemas are defined in an OpenAPI specification maintained with the code.

Method	Endpoint	Purpose
POST	/v1/learners	Create a learner profile
POST	/v1/learners/{id}/consents	Record learner or guardian consent
PUT	/v1/learners/{id}/goal	Set the target role, date and time budget
GET	/v1/learners/{id}/twin	Retrieve learner context (levels, flags, strategy outcomes)
GET	/v1/learners/{id}/skill-graph	Goal sub-graph with per-learner states
GET	/v1/learners/{id}/gaps	Current gaps and active misconceptions
GET	/v1/learners/{id}/roadmap	Current roadmap with "Why this?" reasons

Method	Endpoint	Purpose
POST	/v1/learners/{id}/roadmap/adapt	Regenerate next steps from new evidence
GET	/v1/learners/{id}/today	Today’s mission, including due reviews
POST	/v1/sessions	Start a voice or text session
POST	/v1/assessments	Begin an adaptive assessment or explain-back
POST	/v1/assessments/{id}/responses	Submit a text answer or voice transcript segment
POST	/v1/strategy-outcomes	Record a delayed strategy outcome (internal)
POST	/v1/learners/{id}/corrections	Learner disputes an estimate
POST	/v1/mentor-cases	Open a human-mentor case
POST	/v1/mentor-cases/{id}/feedback	Mentor feedback and outcome
GET	/v1/learners/{id}/export	Export all of the learner’s data
DELETE	/v1/learners/{id}	Erase the learner’s data
POST	/v1/learners/{id}/roadmap/what-if	Preview a roadmap under different hours or deadline
GET	/v1/learners/{id}/roadmap/why-not/{item}	Explain why a topic or resource is not in the roadmap
POST	/v1/learners/{id}/stuck	Diagnose why the learner is stuck (error, query or problem)
POST	/v1/work-products	Submit a query or code for execution and misconception checks
POST	/v1/mentors/applications	Apply to become a mentor (starts verification)
POST	/v1/mentor-cases/{id}/messages	Case-bound message (screened)
POST	/v1/reports	Report a concern about a message or a mentor

Voice-agent tools (exposed to the ElevenLabs agent)

Tool	Purpose
get_learner_context	Minimal context: goal, current concept, level, flags, language
get_next_action	What to do next in this session
explain_topic	Grounded explanation using the selected strategy
record_response	Submit a transcript segment for scoring; returns a follow-up question
request_mentor	Open a mentor case when a trigger fires or the learner asks

No tool can set mastery directly — mastery changes only through graded evidence on the server.

30. Data Flow

1. The learner speaks or types.
2. Voice: the ElevenLabs agent transcribes and manages turn-taking. Text goes straight to the API.
3. The SAARTHI API authenticates the request and loads context from the Learner Twin and Skill Graph.
4. The Diagnostic Engine interprets the response against the concept and the misconception library.
5. The Teaching Strategy Engine picks the next intervention; the Learning Engine generates grounded content.
6. The Assessment Engine scores evidence with a confidence estimate and writes an evidence event.
7. The Adaptation Engine updates concept state, roadmap and review schedule, and checks escalation triggers.
8. The response returns to the learner — spoken through ElevenLabs or shown as text and visuals — with the next action.

9. If triggered, the Mentor Service opens a case; mentor feedback flows back as evidence.

31. Proposed Technology Stack

Layer	Proposed choice	Reason
Frontend	React (Vite) progressive web app	Web and mobile from one codebase; installable
Backend	Python with FastAPI (engine modules adapted from TESSA)	Strong ML ecosystem; async I/O for voice tools; reuses proven planning logic
Database and auth	PostgreSQL via Supabase (auth, row-level security, realtime)	Relational state plus JSON; case messaging; one database instead of two
Cache / jobs	Redis	Session state and background jobs
Voice	ElevenLabs Agents	Speech in/out, turn-taking, Hindi support (to verify in spike)
LLM	Provider-agnostic gateway ported from TESSA's model-pool adapter; default provider **[TO CONFIRM]** (TESSA uses Gemini today)	Fallback across models, response caching and cost caps; no lock-in
Sandbox	SQLite (WebAssembly) and Pyodide in the browser	No server-side code execution
Hosting	Managed cloud, India region	Latency and data-localisation preferences
Observability	OpenTelemetry, error tracking, LLM trace logging	Debugging, cost and quality monitoring

Final choices **[TO CONFIRM]** after the technical spike.

32. Non-Functional Requirements

Area	Initial target (to validate in pilot)
Voice turn latency	Median under 1.0 s and 95th percentile under 2.0 s from end of speech to start of reply
Context-tool latency	95th percentile under 300 ms
Dashboard load	Under 3 s on a mid-range phone over 4G
Availability	99.5% monthly during the pilot
Scale	Pilot: 500 concurrent learners; stateless services scale horizontally
Security	TLS everywhere, encryption at rest, role-based access, audit log, secrets manager, dependency scanning, row-level security on every table, no model or API keys in client code
Cost	Per-learner monthly model and voice budget enforced by the LLM gateway
Accessibility	WCAG 2.1 AA for the web UI; full text parity for all voice features
Localisation	UI in English and Hindi; conversation in English, Hindi and Hinglish

33. Safety, Privacy & Compliance

Principles

- Avoid claiming that voice alone can reliably infer emotion, intelligence or psychological state.
- Keep learner-state updates evidence-based and explainable.
- Provide a way for students to correct inaccurate learner-model assumptions.
- Protect student data and minimise unnecessary storage of sensitive conversation content.
- Clearly distinguish AI-generated guidance from verified academic or career information.

- Keep human mentors in the loop for persistent or high-impact decisions.
- Measure whether adaptive interventions actually improve learning rather than assuming personalisation works.
- No camera or posture analysis: it adds privacy risk (especially for minors) without proven learning value.

Children's data — India's DPDP Act, 2023

Many learners will be under 18. Under the Digital Personal Data Protection Act, 2023, processing a child's data requires verifiable consent from a parent or guardian, and the Act restricts tracking or behavioural monitoring of children and targeted advertising at them; the DPDP Rules set out consent mechanics and certain exemptions (for example, for educational institutions). Because the Learner Twin models learning behaviour, **legal review is required before launching to minors**. Until then, run the pilot with learners aged 18+ or through partner institutions under appropriate agreements.

Data minimisation and retention

- Audio is not stored by default — only transcripts. Transcripts are kept for a limited period (e.g. 90 days **[TO CONFIRM]**) unless the learner opts in; after that, evidence keeps scores, not raw text.
- Self-service export and erasure (see API section).
- Voice, LLM and cloud providers act as data processors under data-processing agreements; zero-retention options and cross-border transfer are reviewed before launch.

Wellbeing and safeguarding

SAARTHI does not infer emotion from voice. If a learner *explicitly* mentions distress, self-harm or abuse, it follows a scripted safe response, shares helpline information (in India, the Tele-MANAS mental-health helpline, 14416) and, for institution learners, alerts the designated safeguarding contact according to that institution's policy.

Mentor safeguarding

Mentor verification, rules for learners under 18, message screening, reporting and the safeguarding lead role are defined in the Human Mentor Layer section. They are launch requirements, not later additions: no mentor sessions open to minors until they are in place and legally reviewed.

Integrity, misuse and fairness

- Explain-backs include live follow-ups; sandbox tasks use randomised data.
- For graded external homework, SAARTHI offers guided help rather than finished answers.
- Prompt-injection resistance: the agent cannot change mastery (see AI Architecture).
- Scores are independent of accent, fluency and language choice. Recognition and grading are tested across accents and English/Hindi/Hinglish, and score gaps between language groups are monitored.

Part C — Delivery & Business

34. MVP Scope

The hackathon build and the first pilot have different requirements, so they are scoped separately. Every step of the recommended demo uses only P0 features. Nothing below has been built yet.

Priority	Feature	Stage
P0	Onboarding with goal (preset Backend Developer template)	Hackathon MVP
P0	Skill graph: SQL and prerequisites (about 30–40 concepts), expert-curated	Hackathon MVP
P0	Placement probe with misconception-mapped questions and a confidence rating	Hackathon MVP
P0	SQL query check: run the learner's queries on seeded data and match wrong-output patterns	Hackathon MVP
P0	Voice and text mentor (ElevenLabs + text channel)	Hackathon MVP
P0	Conversational diagnosis and explain-back with a per-concept rubric	Hackathon MVP
P0	Knowledge-gap **and** misconception detection (curated SQL library, 10–15 entries) with read-back	Hackathon MVP
P0	Unified skill-state model with Bayesian Knowledge Tracing and the evidence log	Hackathon MVP
P0	Adaptive roadmap with "Why this?" (planner ported from TESSA)	Hackathon MVP
P0	Rule-based strategy choice and a basic visual panel (pre-built JOIN diagrams)	Hackathon MVP
P0	Student dashboard	Hackathon MVP
P0	Mentor handoff report (generated and routed; no live mentor matching)	Hackathon MVP
P1	"I'm stuck" button; what-if simulator; "why not X?"	Pilot
P1	Python practice sandbox and spaced review	Pilot
P1	Interview mode (role-based mock interview on the explain-back engine)	Pilot
P1	Learning Genome with "correct me" controls and calibration view	Pilot
P1	Chhatrachhaya: verified Skill Mentors and Margdarshaks, case-bound messaging, safeguarding workflow	Pilot
P1	Guardian consent flow; data export and erasure	Pilot
P1	Teacher / institution cohort view (lite)	Pilot
P1	Teaching-strategy bandit; Hinglish recognition test set; evaluation instrumentation	Pilot
P2	Skill passport; company question packs; resume-based interviews; job matching	Later
P2	Document ingestion; more domains	Later
P2	Mentor marketplace and group clinics; full institutional dashboards	Later

35. Recommended Hackathon Demo

The strongest demo tells one complete story rather than showing dozens of disconnected features. The visual "wow" moment is **the roadmap changing live** after the assessment. The demo now includes the learner's own SQL, which makes the misconception visible without relying on speech recognition alone.

Time	Demo step	P0 feature shown
0:00	Student chooses “Backend Developer” as the target	Onboarding
0:20	SAARTHI displays the initial skill graph	Skill graph
0:40	Student says “I understand SQL joins” and rates themselves “very sure”	Voice mentor, confidence rating
0:55	SAARTHI starts a short voice diagnosis	Conversational diagnosis
1:25	Student runs their own JOIN query; rows are missing; SAARTHI matches the wrong-output pattern	SQL query check
1:50	SAARTHI reads back what it heard — “LEFT JOIN returns only matching rows, right?” — and the student confirms	Misconception detection with read-back
2:10	SAARTHI explains using contrasting cases and a row-level JOIN diagram	Strategy + visual panel
2:50	Student teaches the concept back through voice	Explain-back
3:20	SAARTHI scores the explanation; mastery changes; a calibration note appears (sure, but wrong)	Rubric scoring, unified state model
3:50	The roadmap inserts targeted practice — live, with “Why this?”	Adaptive roadmap
4:30	Simulated persistent failure: SAARTHI prepares a handoff report and routes it to a Skill Mentor	Mentor handoff report

Demo risk plan

- Pre-recorded backup video of the full flow.
- Switch to text mode if the venue network or microphone fails.
- Seeded demo learner account; real API calls rather than mock-ups, and a clear statement of what is live and what is pre-seeded. Nothing is simulated without saying so.

36. Differentiation & Competitive Landscape

Category	Typical focus	SAARTHI focus
Document AI	Understand source material	Use material as one evidence source (Phase 3)
Generic AI tutor	Answer questions	Model learner state
Course platform	Deliver a fixed curriculum	Continuously adapt the curriculum
Voice AI	Natural conversation	Use conversation for diagnosis and assessment
Quiz systems	Measure answers	Infer gaps and misconceptions
Human mentoring	Experience and guidance	AI prepares a contextual escalation

Product	Primary focus	Where SAARTHI differs
Khanmigo (Khan Academy)	Socratic AI tutor over Khan content; teacher tools	Career-oriented self-learners; voice-first Hinglish; persistent, learner-visible skill model
ChatGPT Study Mode, Gemini Guided Learning	Step-by-step guided learning inside general assistants	Structured skill graph and roadmap owned by the learner; explain-back assessment
ALEKS (McGraw Hill)	Adaptive assessment using knowledge space theory	Conversational, open-ended evidence instead of item-only assessment
Duolingo Max	AI conversation for language learning	Technical and career skills
Indian ed-tech apps with AI doubt-solving	Exam preparation and doubt solving at scale	Learner-model-centred rather than answer-centred

Competitor capabilities change quickly; verify before external use.

37. Business Model & Unit Economics

SAARTHI is priced independently of the standalone Chhatrachhaya app, whose promise is to stay free for everyone. Institutions are recommended as the first paying customer: paying for consumer education products is hard, while teachers and training institutes can see immediate value in knowing which misconceptions their cohort shares.

Model	Customer	Notes
Institutional licence (B2B2C) — recommended first	Colleges, bootcamps, coaching institutes	Per-seat annual licence; cohort view (lite in the pilot, full in Phase 5)
B2C freemium	Self-learners	Text mode free with limits; paid plan adds voice minutes and mentor sessions
Mentor sessions	Learners	Volunteer mentors in the pilot; paid sessions with a commission in Phase 4

Cost model

Cost per active learner per month = voice minutes × voice price per minute + LLM tokens × token price + infrastructure share + mentor minutes × mentor cost.

Input	Starting assumption (to validate)
Voice minutes per active learner per month	About 60, used for diagnosis and explain-back only
LLM tokens per session	Measured during the pilot
Voice and LLM unit prices	Vendor pricing at the time of launch
Mentor escalation rate	Under 5% of active learners per month
Pricing and target gross margin	**[TO CONFIRM]** after willingness-to-pay interviews in the pilot

Cost levers: text as the default mode; voice for high-value moments; a smaller model for scoring; caching explanations per concept; per-plan minute caps.

38. Success Metrics & Evaluation Plan

North Star metric: verified mastery gains per active learner per week — concepts that reach Mastered and are still answered correctly at a 7-day review.

Metric	Definition	Pilot target (proposed)
Learning gain	Pre/post test improvement versus a control group	Significantly above control
Misconception resolution	Share of flagged misconceptions still resolved at a 7-day check	≥ 60%
Misconception precision	Share of flags confirmed correct by human review, reported per evidence channel	≥ 90%
Grader agreement	Cohen’s κ between AI rubric scores and human graders	≥ 0.70
Retention	Accuracy on delayed review items	≥ 80%
Learning efficiency	Median time to Mastered per concept	Set baseline, then reduce
Adaptation quality	Share of roadmap changes followed by a better result at the next check	≥ 60%
Roadmap stability	Major re-orders per learner per week	≤ 1

Metric	Definition	Pilot target (proposed)
Mentor handoff usefulness	Mentor rating of the case report (1–5)	≥ 4.0
Mentor follow-up success	Learner's mastery change on the concept after a mentor session	Positive, versus matched cases without a session
Safeguarding response	Reports acknowledged by the safeguarding lead within 24 hours	100%
Engagement	Weekly active learners; 30-day retention	Tracked, never optimised alone
Voice quality	Recognition error rate on the Hinglish test set; voice latency	Baseline first; p95 under 2 s
Cost	Cost per active learner per month	Within plan budget

Validation before building

- Talk to 15–20 learners and 3–5 mentors or teachers: how they get unstuck today, what they would pay for, and whether they would speak to an AI for ten minutes.
- Collect about 60 real learner explanations of SQL joins, label them by hand, and measure how often the model finds the same misconceptions. This number is worth more in a demo than any polished screen.
- Build a small Hinglish technical-speech test set and measure recognition errors before relying on voice.
- Prototype the diagnosis loop in text first; add voice only once the loop works.

Evaluation design

- Pilot with one or two partner institutions **[TO CONFIRM]**.
- Randomised comparison: adaptive SAARTHI versus the same content in a fixed order (both with AI chat), so the effect of adaptation — not just of having an AI — is measured.
- Pre-test, post-test and a 2-week delayed test; metrics defined before the pilot starts.
- Target sample: 100+ learners per group if feasible **[TO CONFIRM]**.
- A human-graded sample of about 200 explain-backs for calibration.

39. Risks & Mitigations

Risk	Impact	Mitigation
Wrong misconception flags	Loss of trust	Work-product evidence first, confirmation question, confidence threshold, precision metric, dispute button
Speech-recognition errors on Hinglish and technical terms	False evidence	Read-back, confidence gating, text fallback, dedicated test set; diagnosis never depends on speech alone
LLM explains something incorrectly	Learner learns wrong content	Grounding in curated notes, error reporting, expert review
Skill-graph quality	Poor roadmaps	Expert curation, versioning, mentor and learner flags
Voice costs higher than planned	Weak margins	Minute caps, text default, model tiering
Dependence on voice or LLM vendor	Outage or price change	Provider-agnostic gateway, text fallback
Children's-data compliance	Legal and regulatory exposure	Legal review, 18+ pilot, guardian consent, data minimisation
Mentor misconduct or unvetted contact with learners	Harm to learners; legal exposure	Verification, case-bound in-platform sessions, message screening, reporting, safeguarding lead
A retired professional is given a technical case	Poor help, wasted trust	Two-tier routing by topic
Too few mentors	Escalations go unanswered	Pilot volunteers, response targets, AI fallback, marketplace later
Prototype shortcuts copied into SAARTHI	Invented numbers, insecure keys, silent failures	Reuse map and design rules; automated tests of the main user path; code review against the lessons table
Personalisation benefits not proven	Credibility	Controlled evaluation before making claims
Gaming or cheating	Invalid mastery	Live follow-ups, randomised tasks, server-side mastery

Riskiest assumptions, in the order to test them

#	Assumption	How to test	When
1	SAARTHI can find misconceptions accurately, including in Hinglish speech	Labelled set of ~60 answers; precision per channel	Step 0
2	Learners will do repeated 10-minute voice sessions	Prototype sessions with 10–15 learners	Step 3 / pilot
3	Changing the roadmap improves results over a fixed order	Randomised pilot with delayed test	Pilot
4	Mentors respond in time and sessions help	Pilot response times; follow-up mastery change	Pilot

40. Future Roadmap

Phase	Capabilities	Priority tier
Phase 0 — Validation	Interviews, labelled evaluation set, Hinglish voice test, SQL graph and misconception library drafts	Before build
Phase 1 — Hackathon MVP	Learner Twin, SQL skill graph, voice and text diagnosis, query check, gap and misconception detection, explain-back, adaptive roadmap, mentor handoff report	P0
Phase 2 — Pilot	I'm stuck, what-if, why-not, interview mode, strategy bandit, practice sandbox, spaced review, Learning Genome controls, verified mentors and safeguarding, cohort view (lite), consent flow	P1
Phase 3 — Career & content	Skill passport, company packs, resume-based interviews, job matching, document ingestion	P2
Phase 4 — Chhatrachhaya at scale	Mentor marketplace, paid sessions, group clinics, mentor quality ratings	P2
Phase 5 — Institutions	Full teacher and institution dashboards, aggregate skill intelligence, curriculum alignment	P2
Phase 6 — Cross-domain	More domains and Indian languages; long-term learning memory across goals	Later

41. Build Plan and Pre-Build Decisions

> **Status: not started.** No application code has been written. The steps below are a proposal, to begin only after this report is approved.

Step	Scope	Reuses (ideas, re-implemented)	Exit criteria	Effort
0 Validate	Interviews; labelled evaluation set; Hinglish voice test; draft SQL graph and misconception library	—	Go / no-go on detection accuracy; content owner confirmed	M
1 Foundation	Repository, database schema with row-level security, authentication, LLM gateway, skill-graph service, Bayesian Knowledge Tracing service, evidence log	llm; store (replaced); profiler (replaced)	Unit tests for graph and mastery model; security verified	M
2 Core loop in text	Diagnostic engine with misconception-mapped items, SQL query check, explain-back with rubric, roadmap with "Why this?"	planner; gap; explainer; diagnostic; interview	Full loop works end to end in text for a seeded learner	L
3 Voice and visual panel	ElevenLabs agent with server tools, read-back, visual panel, text parity	chat (pattern)	Demo script runs live in voice and text	M
4 Dashboard and handoff	Student dashboard, Genome (lite), mentor case-report generation and routing	—	Handoff report reviewed by a real mentor	M
5 Demo hardening and first evaluation	Backup video, seeded learner, metrics instrumentation, small pre/post test	—	Rehearsed demo; first precision and agreement numbers	M

Step	Scope	Reuses (ideas, re-implemented)	Exit criteria	Effort
6 Pilot build (P1)	Mentor Service with verification and safeguarding, console, sandbox, interview mode, cohort view (lite), strategy bandit	Chhatrachhaya concepts	Safeguarding and legal review passed; pilot-ready	L

Decisions needed before build starts

1. Confirm the mentor layer: the Chhatrachhaya name, the two tiers (Skill Mentor and Margdarshak) and the safeguarding lead role.
2. Confirm the stack: React (Vite) PWA, FastAPI, Supabase (PostgreSQL, authentication, realtime).
3. Name the owner of the first-domain content (SQL graph, questions, misconception library, rubrics).
4. Choose the default LLM provider for SAARTHI (TESSA uses Gemini today; the gateway stays provider-agnostic).
5. Choose the voice mode: ElevenLabs agent with server tools, or SAARTHI's own model endpoint.
6. Decide the pilot age policy (18+ only, or minors through institutions).
7. Set the hackathon date and any time limit, so scope can be cut to fit.
8. Confirm that TESSA modules are copied and adapted rather than imported, keeping SAARTHI independent.

42. Team, Milestones & Open Questions

Role	Owner
Product lead	**[TO CONFIRM]**
AI / ML engineer	**[TO CONFIRM]**
Backend engineer	**[TO CONFIRM]**
Frontend engineer	**[TO CONFIRM]**
Domain expert (SQL / backend)	**[TO CONFIRM]**
Design	**[TO CONFIRM]**
Mentor programme and safeguarding lead	**[TO CONFIRM]**

Milestone	Definition of done	Date
M0 — Validated	Interviews done; evaluation set labelled; voice test run; go/no-go recorded	**[TO CONFIRM]**
M1 — Content ready	SQL skill graph, item bank, rubrics and misconception library reviewed	**[TO CONFIRM]**
M2 — Core loop in text	Diagnosis → teach → explain-back → assess → adapt works end to end in text	**[TO CONFIRM]**
M3 — Voice integrated	ElevenLabs agent calling SAARTHI tools; read-back safeguards working	**[TO CONFIRM]**
M4 — Demo-ready MVP	Full demo script runs live, with backup plan rehearsed	**[TO CONFIRM]**
M5 — Pilot launch	Consent flow, verified mentors, safeguarding and evaluation instrumentation live	**[TO CONFIRM]**

Open questions

- First user segment: college students or working professionals switching careers?

- Age policy for the pilot (18+ only, or minors through institutions)?
- Voice integration mode: ElevenLabs-hosted LLM with tools, or SAARTHI's own LLM endpoint?
- Pilot institution partners and mentor recruitment sources.
- Pricing and plan structure.
- Whether police verification is required for mentors who meet minors (legal advice).

43. Product Positioning & Final Statement

SAARTHI should not be positioned as "another AI tutor", "a voice-enabled study assistant" or "an AI PDF reader". Those descriptions reduce the product to individual features.

> **Recommended positioning:** SAARTHI is an Adaptive AI Mentor that builds a living model of the learner and continuously decides what the learner should learn, why they are struggling, how to teach the concept, and when human guidance is needed.

First wedge: "practise explaining". Explain-back is what viva and interview rounds test, so it gives learners an immediate reason to use SAARTHI, brings in career intelligence naturally, and gives schools and institutes a clear reason to pay.

SAARTHI transforms learning from a static content-delivery process into a continuous learner-modelling loop. It does not simply ask "What should I teach?" It asks: **"What does this learner understand, what is preventing progress, what teaching approach is working, and what should happen next?"**

SAARTHI — The AI Mentor That Learns How to Teach You.

Understand → Diagnose → Teach → Listen → Assess → Adapt → Repeat

Appendix — Glossary

Term	Meaning in this report
Learner Twin	SAARTHI's internal, structured, evidence-linked model of one learner
Learning Genome	The learner-facing, plain-language view of the Learner Twin
Skill graph	Concepts (nodes) and prerequisite relationships (edges) for a domain
Mastery level	Not assessed, Unknown, Developing, Proficient or Mastered — derived from P(known)
Flag	Independent marker on a concept: Misconception, Blocked by prerequisite, Due for review, Disputed, Low confidence
Misconception	An incorrect mental model, as opposed to missing knowledge
Evidence event	One recorded, scored observation of the learner; the basis for every state change
Explain-back	Assessment in which the learner teaches the concept back in their own words
Teaching strategy	A way of teaching a concept, e.g. analogy, worked example, contrasting cases
Chhatrachhaya	SAARTHI's human mentor network for escalated cases
Mentor case	A structured handoff containing goal, evidence, strategies tried and suggested next step
BKT	Bayesian Knowledge Tracing — the probabilistic model used to estimate P(known)
Margdarshak	An experienced or retired professional mentor in Chhatrachhaya (career, interview and guidance cases)
Skill Mentor	A working developer, senior student or assistant who takes technical cases
Safeguarding lead	Trained staff member who handles reports of distress, abuse or misconduct
Work product	Something the learner produces, such as a SQL query, that can be checked by execution
Calibration	How well a learner's stated confidence matches their actual results
Evidence provenance	The recorded source type of each evidence event (question, work product, explain-back, mentor, feedback)
TESSA	The separate adaptive-learning and placement-readiness project whose planning ideas SAARTHI re-implements
P0 / P1 / P2	Priority tiers: hackathon MVP / pilot / later